

SkillSphere Project Demo Guide

Project review date: 03 June 2026

Project: SkillSphere - Intelligent Internship, Jobs, Courses, and Freelance Marketplace

Stack: React, Tailwind CSS, Framer Motion, Node.js, Express.js, MongoDB/Mongoose, Socket.IO, JWT

1. What SkillSphere Is

SkillSphere is a complete career ecosystem for students, interns, freelancers, companies, and learners. It combines internships, job applications, freelance projects, course enrollments, real-time collaboration, payments, review management, and admin oversight into one unified web application.

This application is designed to support:

- Internship discovery, application, and management
- Job posting and candidate matching
- Freelance project listing and proposal workflows
- Paid course enrollment and learning records
- Real-time messaging, notifications, and collaboration
- Admin monitoring of users, gigs, payments, disputes, and revenue

2. What We Built and Why It Matters

Core product features

- Role-based registration and login for clients, freelancers, and admins
- Secure JWT authentication with protected routes
- Freelancer profile management with skills, portfolio, availability, pricing, and reputation
- Marketplace listings for projects, internships, jobs, and courses
- Proposal and application flow for freelancers and interns
- Payment flow supporting escrow-style checkout and transaction tracking
- Real-time chat and notification system using Socket.IO
- Admin dashboard with analytics, user controls, and review/dispute oversight

Why this fulfills the requirement

- The app is not just a job board; it is a multi-faceted internship and career platform.
- It includes internships, jobs, freelance work, and course payments in one UX.
- It includes the backend architecture expected for a production-ready MERN app.
- It supports role-based data flows, provider-ready integrations, and demo-safe operation.
- It preserves the existing UI while adding full functionality and validation.

3. How We Fulfilled Their Requirements

Authentication and Access

- Implemented registration and login with JWT.
- Added client/freelancer/admin roles with protected route enforcement.
- Added email verification token flow and password reset token flow.
- Added demo two-factor authentication and Google login endpoint.
- Added provider-ready backend routes for external integrations.

Marketplace and Matching

- Added gig listing support with type categories: projects, internships, jobs, and courses.
- Added advanced listing details: skills, duration, location, pay, deadline.
- Added proposal/application submission, negotiation, and status updates.
- Added skill matching logic with score weighting for matches.
- Added trending skills and recommendation data.

Profile and Portfolio

- Added profile fields: skills, expertise, portfolio items, resume link, certifications, experience.
- Added hourly and milestone pricing, availability slots, and verification badge metadata.
- Added reputation score and profile view tracking.

Collaboration and Communication

- Added Socket.IO real-time messaging.
- Added notifications for new messages, proposals, payments, and status changes.
- Added conversation history and review submission.

Payments and Escrow

- Added payment ledger and records, including pending, released, and refunded states.
- Added provider-ready checkout support for Stripe and Razorpay.
- Added mock provider fallback so the app can demo without live payment keys.
- Added platform fee calculation and payment confirmation.

Reviews and Disputes

- Added verified review handling with weighted reputation.
- Added dispute creation, evidence metadata, admin notes, and resolution status.
- Added admin access for dispute handling and platform safety.

Admin Oversight

- Added admin dashboard with total users, payments, revenue, and gig activity.
- Added user role updates and platform analytics.
- Added admin logs for key actions.

4. Technologies Used

Frontend

- React 19
- Vite

- Tailwind CSS
- Framer Motion
- React Router DOM
- Axios
- React Hot Toast
- Socket.IO client

Backend

- Node.js + Express
- MongoDB + Mongoose
- Socket.IO server
- JSON Web Tokens (JWT)
- bcrypt for passwords
- Nodemailer provider-ready email support
- Cloudinary upload support
- Stripe and Razorpay provider-ready payment integration
- Redis readiness package support

Deployment and Environment

- '.env' support for 'MONGO_URI', 'JWT_SECRET', 'CLIENT_URL', provider keys, and feature flags
- CORS configured for the frontend origin
- Rate limiting and security headers
- Build-ready frontend and production preview

5. What We Fixed and Verified

- Fixed Google login callback issues in the login page.
- Verified the client build completes successfully.
- Verified the backend starts cleanly and responds to '/api/health'.
- Verified the built frontend preview serves successfully on 'http://127.0.0.1:4173'.
- Preserved the existing UI and design while wiring functional backend support.
- Prepared a structured demo guide for a 5-minute screen-share presentation.

6. Screen-Share Presentation Plan (5 Minutes)

1. Start with the elevator pitch

"SkillSphere is a combined internship, job, freelance, and course marketplace built for students and small companies."

2. Landing page

- Show the product positioning and UI.
- Point out the search, AI match, stats, and service cards.

3. Register/Login

- Show role selection on register.
- Show secure JWT login and protected access.

4. Marketplace

- Show internships, jobs, projects, and courses.
- Post a listing and submit a proposal.

5. Profile

- Show freelancer profile details and pricing.

6. Payments

- Show payment ledger and transaction summary.
- Explain provider-ready Stripe/Razorpay integration.

7. Admin overview

- Show admin analytics and user controls.

8. Close with readiness

- Emphasize that the platform is ready for internship review and can be extended with live provider keys.

7. Interview Notes

- Emphasize that the platform is not just a job board but a full career ecosystem.
- Mention provider-ready integration: Stripe/Razorpay, Google OAuth, Cloudinary, SMTP.
- Highlight backend architecture: Express API, MongoDB data models, role-based middleware, Socket.IO collaboration.
- Highlight frontend UX: responsive Tailwind design, animated transitions, consistent branding.
- Note that live payment/email providers need keys, while demo flows work without them.

8. Running the App

Backend

```
""bash
cd server
npm install
npm run dev
""
```

Frontend

```
""bash
cd client
npm install
npm run dev
""
```

Built preview

```
""bash
```

```
cd client
npm run build
npm run preview -- --host 127.0.0.1 --port 4173
""
```